Name: Kedar Somnath Dhotre

PRN: 23510090

Experiment no 6

Assignment 6: Introduction to MPI : Installation, Processes, Ranks, Groups and Point-to-Point Communication

Aim:

To configure the MPI environment and understand the fundamentals of distributed-memory parallel programming by creating MPI processes, identifying process ranks, working with MPI groups, and implementing one-to-one communication using MPI_Send() and MPI_Recv().

Why Are We Learning This?

OpenMP allows multiple threads to work in a shared memory space. However, many HPC applications run on clusters where each computing node has its own memory.

MPI (Message Passing Interface) provides a standard programming model for such distributed-memory systems.

In MPI:

- A program is executed by multiple processes.
- Each process has its own address space.
- Processes communicate by exchanging messages.
- Each process is identified by a unique rank.
- Processes can be organized into groups and communicators.

MPI is widely used in:

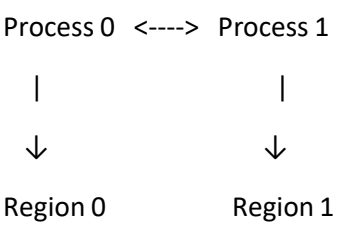
- Weather and climate simulation
- Computational fluid dynamics
- Molecular dynamics
- Large-scale scientific simulations
- Distributed AI/ML workloads
- Big-data processing
- Supercomputing applications

HPC in Practice

Consider a weather simulation running on 100 computing nodes.

Each process may calculate the weather conditions for a particular geographical region. However, neighbouring regions need to exchange information.

For example:



MPI provides the communication mechanism required to exchange this information.

If each process has its own memory:

How can Process 0 send data to Process 1?

The answer is message passing.

This assignment introduces the basic mechanism using:

MPI_Send()

MPI_Recv()

Learning Focus

This assignment introduces:

- MPI environment
- MPI program structure
- MPI initialization and termination
- MPI processes
- MPI rank
- MPI size
- MPI communicator
- MPI groups
- Point-to-point communication
- MPI_Send()
- MPI_Recv()
- Message tags

- Source and destination
- Basic performance measurement

Pre-Lab Activity

Answer the following questions before starting the assignment.

1. What is distributed-memory parallel computing?

   a) Distributed-memory parallel computing is a type of parallel computing in which multiple processors or computers have their own separate local memory.

   b) Each processor works on a part of the problem simultaneously.

   c) The processors cannot directly access each other's memory.

   d) They communicate with each other by sending and receiving messages through a communication network.

   e) It is useful for solving large and complex problems using multiple computers or nodes.

   f) MPI is commonly used for programming distributed-memory systems.

2. What is MPI?

   a) MPI stands for Message Passing Interface.

   b) It is a standardized message-passing library/interface used for parallel and distributed computing.

   c) MPI allows different processes to communicate and exchange data with each other.

   d) It is commonly used in distributed-memory systems.

   e) MPI supports languages such as C, C++, Fortran, Java and Python through different implementations/libraries.

3. What is the difference between a process and a thread?

Process	Thread
A process is a program in execution.	A thread is a smaller unit of execution within a process.
Each process has its own memory space.	Threads of the same process share the same memory space.
Processes are generally more resource-intensive.	Threads are generally lighter and faster than processes.
Communication between processes requires mechanisms such as message passing or IPC.	Threads can communicate easily by using the shared memory of their process.
One process can contain multiple threads.	A thread always belongs to a process.

4. What is an MPI rank?
    a) An MPI rank is a unique number/identifier assigned to each MPI process within a communicator.
    b) It is used to identify a particular process during communication.
    c) Ranks normally start from 0.
    d) For example, if there are 4 processes, their ranks will be:
    e) Process 1 → Rank 0
    f) Process 2 → Rank 1
    g) Process 3 → Rank 2
    h) Process 4 → Rank 3
    i) MPI_Comm_rank() is used to find the rank of the calling process.

5. What is an MPI communicator?
    a) An MPI communicator is an object that defines a group of MPI processes that can communicate with each other.
    b) It provides the communication context in which MPI operations take place.
    c) A communicator contains information about the process group and their ranks.
    d) MPI functions such as MPI_Send() and MPI_Recv() use a communicator to specify the communication group.
    e) The most commonly used communicator is MPI_COMM_WORLD.
6. What is the purpose of MPI_COMM_WORLD?
    a) MPI_COMM_WORLD is the default communicator provided by MPI.
    b) It includes all MPI processes that are part of the MPI program.
    c) Each process in MPI_COMM_WORLD gets a unique rank from 0 to number of processes - 1.
    d) It is commonly passed to functions such as:
        a. MPI_Comm_size()
        b. MPI_Comm_rank()
        c. MPI_Send()
        d. MPI_Recv()
    e) For example, if we run a program using 4 processes, MPI_COMM_WORLD contains all 4 processes with ranks 0, 1, 2 and 3.

7. What is the difference between MPI_Send() and MPI_Recv()?

MPI_Send()	MPI_Recv()
Used to send a message/data from one process to another.	Used to receive a message/data from another process.
The sender specifies the destination rank.	The receiver specifies the source rank.
It takes a send buffer containing the data.	It takes a receive buffer where the received data is stored.
Example: MPI_Send(...)	Example: MPI_Recv(...)
It performs a standard-mode blocking send.	It performs a standard blocking receive.

8. Why is message passing necessary in distributed-memory systems?
    a) In distributed-memory systems, each processor/node has its own separate memory.
    b) One process cannot directly access the memory of another process.
    c) Therefore, processes need a method to exchange data and information.
    d) Message passing allows one process to send data to another process.
    e) It helps processes coordinate their work and combine their individual results.
    f) It allows a large problem to be divided into smaller tasks and processed by different nodes.
    g) MPI provides functions such as MPI_Send() and MPI_Recv() for this communication.
    h) Therefore, message passing is essential for communication and cooperation between processes in distributed-memory systems.

Section A – Basics

Task 1: MPI Installation and Environment Verification

Install and configure an MPI implementation such as **OpenMPI** or **MPICH**.

Verify the installation using appropriate commands.

For example:

mpicc --version

and:

mpirun --version

or:

mpiexec --version

Record:

- MPI implementation
- MPI version
- C compiler version
- Operating system
- CPU configuration

Task 2: First MPI Program

Write an MPI program to display:

Hello from MPI

The program must contain the appropriate MPI initialization and termination routines.

Identify the purpose of:

MPI_Init()

MPI_Finalize()

Compile and execute the program using:

mpicc hello.c -o hello

mpirun -np 4 ./hello

Task 3: Understanding MPI Rank and Size

Modify the program to display:

- Process rank
- Total number of processes

Use:

MPI_Comm_rank()

MPI_Comm_size()

Execute using:

- 2 processes
- 4 processes
- 8 processes

Expected Output

The order of output may vary.

Process 0 of 4

Process 1 of 4

Process 2 of 4

Process 3 of 4

Record Your Observation

Number of Processes	Observed Ranks	Total Processes
2	**1, 0**	**2**
4	**0, 1, 2, 3**	**4**
~~8~~	**~~0, 2, 1, 6, 5, 4, 7, 3~~**	**~~8~~**

Task 4: Understanding MPI Groups and Communicators

Investigate the relationship between:

MPI_COMM_WORLD → MPI Group → MPI Communicator

Using MPI group-related functions, create a subgroup containing selected processes from MPI_COMM_WORLD.

For example, if there are 8 processes:

MPI_COMM_WORLD

Rank: 0 1 2 3 4 5 6 7

↓ ↓ ↓ ↓

Selected: 0 2 4 6

Create a subgroup containing selected rnks and explore:

- MPI_Comm_group()
- MPI_Group_incl()
- MPI_Comm_create_group()

Display the rank of each process in the newly created communicator.

Global Rank	**Rank within New Communicator**
0	0
2	1
4	2
6	3

Important

Understand the difference between:

Global rank

and

Rank within a communicator.

Global rank is the rank of a process in MPI_COMM_WORLD.
Rank within a communicator is the rank assigned to that process in the newly created communicator.

Think Like an HPC Programmer

Consider 16 processes running on an HPC cluster.

Suppose:

- Processes 0–7 perform one computational task.
- Processes 8–15 perform another computational task.

Answer:

1. Why might it be useful to create separate communicators?

   Separate communicators allow processes working on different tasks to communicate independently.
   For example:
   - Processes 0–7 can communicate using one communicator.
   - Processes 8–15 can communicate using another communicator.

   This avoids unwanted communication between the two tasks and makes the program easier to organize.

2. Can the same process belong to different communication groups?

   A process can belong to multiple groups and communicators at the same time.

   For example, a process could belong to:

   - A communicator for its computational task.
   - Another communicator for communication with a larger set of processes.

3. Why is the communicator important in MPI communication?

   A communicator defines **which processes can communicate with each other and the communication context**.

   MPI communication functions such as MPI_Send() and MPI_Recv() use a communicator to identify the communication group.

   It also allows the same rank number to exist safely in different communicators without confusion.

4. How can groups help organize large parallel applications?

Groups help divide a large set of processes into smaller logical teams based on their tasks.

For example:

16 Processes

|

├— Processes 0–7 → Task 1 → Communicator 1

|

└— Processes 8–15 → Task 2 → Communicator 2

This makes large parallel programs more organized, modular, and easier to manage, because each group can perform its own computation and communication independently.

Section B – Application: One-to-One Communication

Task 5: MPI Send and Receive

Implement a program in which **Process 0 sends a message to Process 1**.

Process 1 should receive the message and display it.

Use:

MPI_Send()

MPI_Recv()

Task 6: Integer Data Communication

Modify the previous program so that Process 0 sends an integer value to Process 1.

Example:

Process 0:

Value = 100

MPI_Send()

↓

Process 1:

Received Value = 100

Verify the received value.

Task 7: Array Communication

Extend the program so that Process 0 sends an array of integers to Process 1.

For example:

Process 0

10 20 30 40 50

|

| MPI_Send()

↓

Process 1

10 20 30 40 50

Process 1 should calculate the:

- Sum
- Average
- Maximum

of the received array.

Performance Activity

Measure the communication time for different message sizes.

Test with:

- 1 KB
- 10 KB
- 100 KB
- 1 MB

- 10 MB

Record:

Message Size	Communication Time
1 KB	0.000007880 seconds
10 KB	0.000014114 seconds
100 KB	0.000013058 seconds
1 MB	0.000104152 seconds
10 MB	0.002746909 seconds

Analyze

1. How does communication time change with message size?

   Communication time generally **increases as the message size increases**. In our experiment, 1 KB took 0.000007880 seconds, while 10 MB took 0.002746909 seconds. The 100 KB result is slightly lower than 10 KB, which can happen because of system scheduling, buffering, and measurement overhead.

2. Why is communication overhead important in distributed-memory computing?

   In distributed-memory computing, processes have separate memory and need to exchange data using messages. This communication takes time. Therefore, excessive communication can reduce the overall performance of a parallel program.

3. Why might MPI communication become a bottleneck?

   MPI communication can become a bottleneck when processes exchange large amounts of data or communicate very frequently. Processes may spend time waiting for messages instead of performing computation. Network latency, bandwidth, and synchronization can also affect performance.

Challenge: Communication

Implement a **communication** between Process 0 and Process 1.

Requirement

Process 0 sends a message to Process 1.

Process 1 sends the message back.

This continues for a specified number of iterations.

Process 0 Process 1

Send ---------->



Measure the total communication time. Calculate:

Round-Trip Time

$$RTT = \frac{Total\ Time}{Number\ of\ Iterations}$$

Investigate how RTT changes with message size.

RTT generally increases as the message size increases.
From the experiment:

- 1 KB: 0.000000784 s
- 10 KB: 0.000004980 s
- 100 KB: 0.000029442 s
- 1 MB: 0.000132617 s
- 10 MB: 0.002312875 s

The 10 MB message has a much larger RTT than the 1 KB message because significantly more data has to be transferred during each round trip.

Advanced Challenge: Avoiding Communication Problems

Modify the program to investigate the behavior of blocking communication.

Consider:

MPI_Send()

MPI_Recv()

Study what happens if both processes attempt to send data before receiving.

Explain:

- Why deadlock can occur.

  MPI_Send() and MPI_Recv() are blocking operations.

  In this program:

  Process 0 → MPI_Send() → MPI_Recv()
  Process 1 → MPI_Send() → MPI_Recv()

  Both processes enter MPI_Send() before either reaches MPI_Recv().

  For the large 10 MB messages, both sends became blocked. Since neither process reached its receive operation, the messages could not be completed.

  This circular waiting condition results in **deadlock**.

- How message ordering affects communication.

  he order of MPI_Send() and MPI_Recv() is important.

  Unsafe ordering:

  Process 0: Send → Receive
  Process 1: Send → Receive

Both processes can become blocked in MPI_Send().

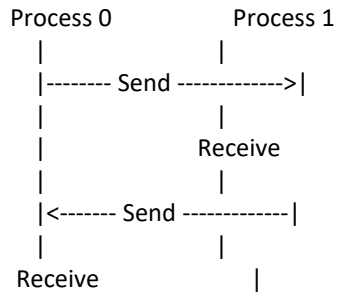
Safe ordering:

Process 0: Send → Receive
Process 1: Receive → Send

Now Process 1 receives Process 0's message before trying to send its response.

- How communication can be redesigned to avoid deadlock.

  he communication can be redesigned using a proper send/receive ordering:



  Another solution is to use:

  MPI_Sendrecv()

  which performs sending and receiving together.

  Non-blocking communication can also be used:

  MPI_Isend()
  MPI_Irecv()
  MPI_Wait()

  This allows communication to be initiated without immediately blocking the process.

Do not deliberately run an unbounded deadlocking program on a shared HPC system. Use a controlled small test and terminate it safely if required.

HPC / AI Case Study

Choose **one** application:

- Weather forecasting

- Computational fluid dynamics
- Molecular dynamics
- Distributed deep learning
- Large-scale scientific simulation
- Graph processing
- Image processing

Identify:

1. Why is distributed computing required?
2. What data needs to be exchanged between processes?
3. Which processes need to communicate?
4. How could MPI be used?
5. What would happen if communication time becomes larger than computation time?

Write a **300–500 word case study**.

Weather Forecasting Using the Weather Research and Forecasting (WRF) Model

The Weather Research and Forecasting (WRF) model is a real high-performance computing application used for atmospheric research and weather forecasting. WRF was developed through collaboration involving organizations such as NCAR and NOAA and is designed to run efficiently on parallel computing systems. A published study also used WRF to investigate the 26 July 2005 heavy rainfall event in Mumbai, which recorded 944 mm of rainfall in 24 hours.

1. Why is distributed computing required?

Weather forecasting involves solving numerical equations for atmospheric conditions such as temperature, pressure, wind, humidity and rainfall over a large geographical region. A high-resolution simulation contains a very large number of grid points and requires substantial computation. Therefore, the geographical domain can be divided into smaller subdomains and processed simultaneously by multiple processors. WRF is designed for parallel execution and uses MPI for distributed-memory parallelism.

2. What data needs to be exchanged between processes?

Each process calculates its assigned geographical subdomain but also needs information from neighboring subdomains. Processes therefore exchange boundary or halo data, such as temperature, pressure, wind, humidity and other atmospheric variables. Boundary and

initial-condition information is also required during the simulation.

3. Which processes need to communicate?

Processes responsible for neighboring geographical subdomains need to communicate with each other. For example, a process handling one grid region exchanges boundary values with the processes handling the regions immediately north, south, east and west of it. This allows the entire atmospheric simulation to remain consistent.

4. How could MPI be used?

MPI can assign different geographical subdomains to different processes. Functions such as MPI_Send(), MPI_Recv(), MPI_Isend() and MPI_Irecv() can be used to exchange boundary data between neighboring processes. Published research on WRF specifically investigates MPI process distribution and its effect on performance.

5. What happens if communication time becomes larger than computation time?

If processes spend more time exchanging data than performing calculations, the overall performance decreases. Some processes may finish their computation and remain idle while waiting for data from neighboring processes. This communication bottleneck limits scalability. Research on large-scale weather models has identified communication and load balancing as important performance issues.

Thus, distributed computing and MPI allow WRF to perform high-resolution weather simulations by dividing the workload among many processors while efficiently exchanging the data required between neighboring regions.

Real case-study sources

- Analysis of the 26 July 2005 heavy rain event over Mumbai using WRF — Wiley
- Analysis of a New MPI Process Distribution for the WRF Model — Scientific Programming
- Weather Forecasting using the WRF model on Google Cloud — Google Cloud

Questions

Basic

1. What is MPI?
   MPI (Message Passing Interface) is a standard used for communication between processes in a distributed-memory parallel system. It allows processes to **send and receive data**.

2. What is an MPI process?

   An **MPI process** is an independent instance of a program running as part of an MPI parallel application. Each process has its **own memory space**.

3. What is an MPI rank?

   An **MPI rank** is a unique integer ID assigned to each process within a communicator. Ranks normally start from **0**.

4. What is the purpose of MPI_COMM_WORLD?

   MPI_COMM_WORLD is the **default communicator** created by MPI. It contains **all processes** participating in the MPI program.

5. What is the difference between MPI_Send() and MPI_Recv()?

MPI_Send()	**MPI_Recv()**
Sends data	Receives data
Used by sender process	Used by receiver process
Specifies destination	Specifies source

Moderate

6. Differentiate between an MPI group and communicator.

   MPI Group: A collection of MPI processes.

   MPI Communicator: Defines a group of processes along with a **communication context** used for communication.

   Simple:

   Group = who is included
   Communicator = who can communicate and in which communication context

7. Why can two processes have different ranks in different communicators?

   Because **rank is relative to the communicator**.

   For example, a process may have:

   MPI_COMM_WORLD rank = 4
   New communicator rank = 2

The process itself has not changed; only its rank within that communicator has changed.

8. What information is specified when using MPI_Send()?

   MPI_Send() specifies:

   MPI_Send(buffer, count, datatype, destination, tag, communicator);

   It specifies:

   - **buffer** – data to send
   - **count** – number of elements
   - **datatype** – type of data
   - **destination** – receiving process rank
   - **tag** – message identifier
   - **communicator** – communication context

9. What information is specified when using MPI_Recv()?

   MPI_Recv(buffer, count, datatype, source, tag, communicator, status);

   It specifies:

   - **buffer** – where received data is stored
   - **count** – maximum number of elements
   - **datatype** – type of data
   - **source** – sending process rank
   - **tag** – message identifier
   - **communicator** – communication context
   - **status** – information about the received message

10. Why does the order of send and receive operations matter?

    Because blocking communication can make a process **wait** until the required communication can complete.

    For example, if both processes do:

    Send → Receive

    both may become stuck in Send(), causing **deadlock**.

    A proper ordering such as:

Process 0: Send → Receive
Process 1: Receive → Send

can avoid this problem.

Advanced

11. What is communication overhead?

    Communication overhead is the time and resources spent exchanging data between processes instead of performing computation.

    It includes things such as message transmission, synchronization and waiting.

12. Explain how message size affects MPI performance.

    As **message size increases**, more data has to be transferred, so communication generally takes more time.

    For example, in your experiment:

    1 KB → very small communication time
    10 KB → higher
    1 MB → higher
    10 MB → much higher

    Large messages can therefore increase communication overhead and reduce performance.

13. What is deadlock in MPI?

    Deadlock occurs when processes wait indefinitely for each other and cannot continue execution.

    Example:

    Process 0 → Send → waiting
    Process 1 → Send → waiting

    If neither reaches its Receive(), both processes remain blocked.

14. Why can blocking communication lead to deadlock?

    Blocking operations do not allow the process to continue until the communication operation can complete.

    For example:

Process 0 Process 1

| |

Send() Send()

| |

WAIT WAIT

| |

Receive() Receive()

Both processes may wait for the other process to receive the message, so neither can proceed.

15. Why is communication an important consideration when designing distributed-memory algorithms?

In distributed memory, processes have separate memory, so they must communicate to exchange required data.

If communication is excessive or slow:

- Processes spend more time waiting.
- Communication overhead increases.
- CPU resources may remain idle.
- Overall execution time increases.
- Parallel scalability decreases.

Therefore, a good distributed-memory algorithm should minimize unnecessary communication and organize communication efficiently.

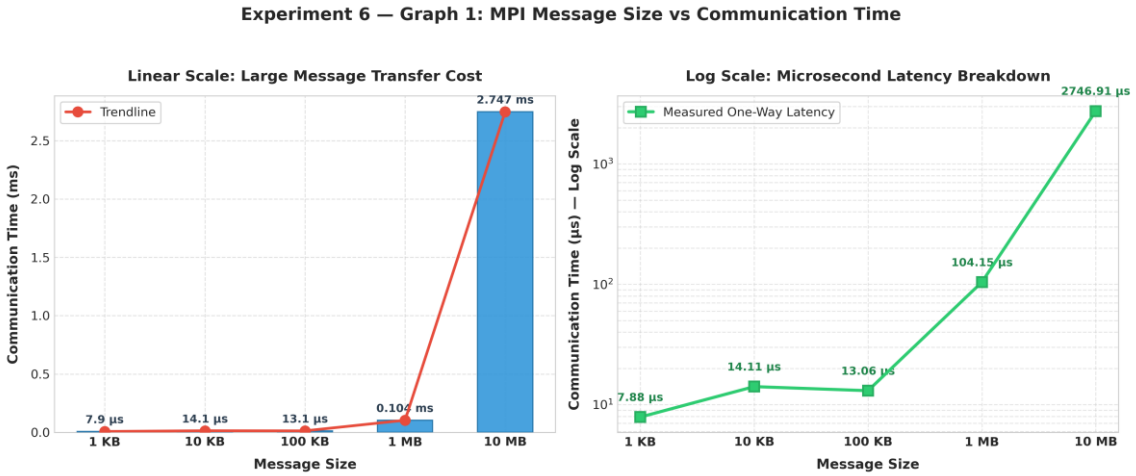
Performance Analysis

For the ping-pong experiment, prepare:

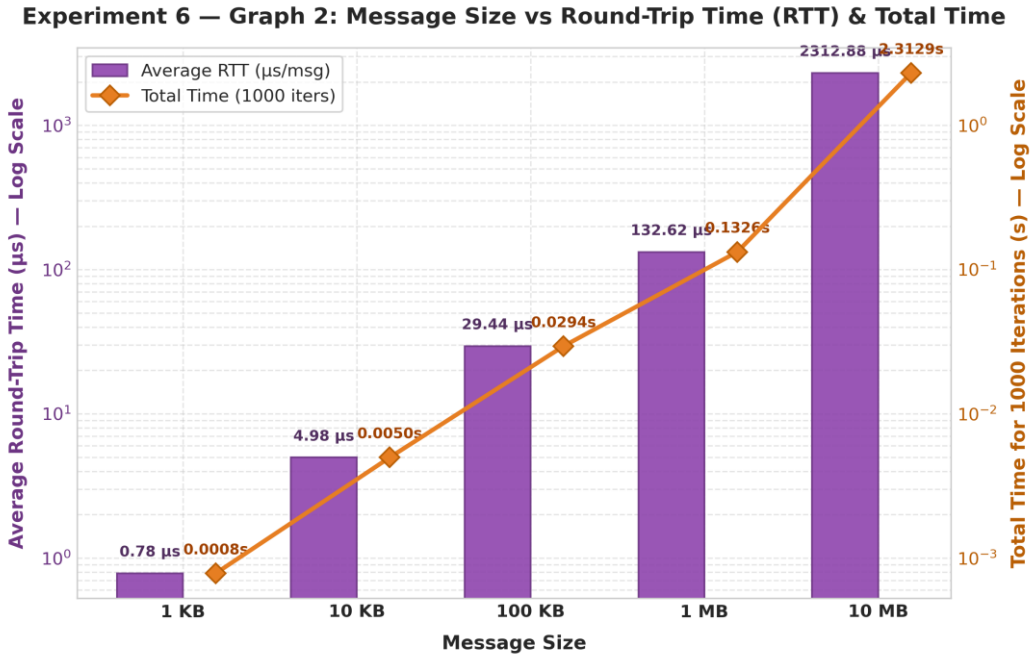
Message Size	Iterations	Total Time (s)	Average RTT (s)
1 KB	1000	0.000784011	0.000000784
10 KB	1000	0.004979743	0.000004980
100 KB	1000	0.029441544	0.000029442
1 MB	1000	0.132617161	0.000132617
10 MB	1000	2.312875289	0.002312875

Plot:

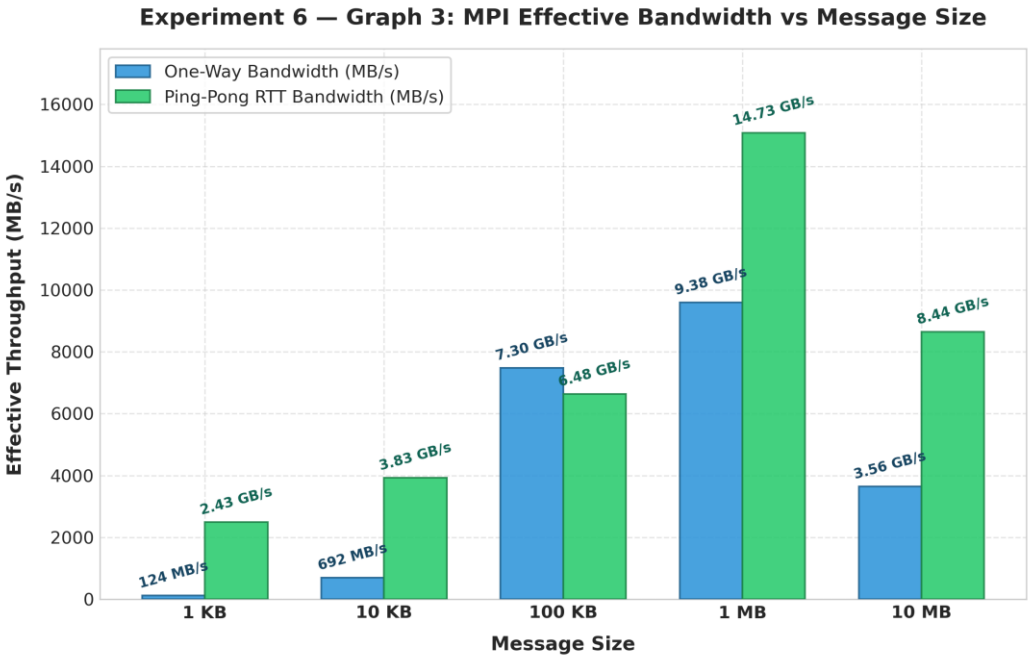
1. **Message Size vs Communication Time**



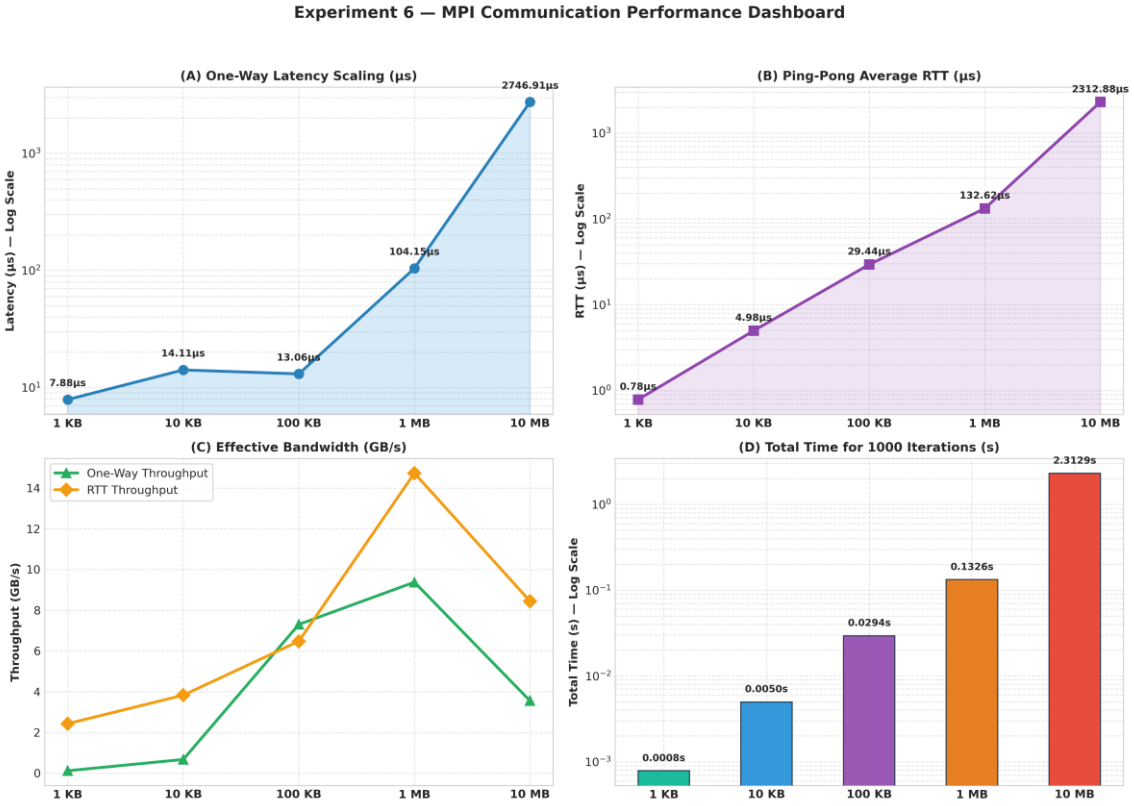
2. **Message Size vs Round-Trip Time**



Message Size vs Effective Bandwidth



MPI Communication Performance Summary Dashboard



Discuss the observed trend.

- The communication time increases as the message size increases.
- For 1 KB, the total communication time is 0.000784011 s, while for 10 MB it increases to 2.312875289 s.
- The average Round-Trip Time (RTT) also increases with message size, from 0.000000784 s for 1 KB to 0.002312875 s for 10 MB.
- The increase is because larger messages require more data to be transferred between MPI processes, resulting in higher communication overhead.
- The increase is especially noticeable for the 10 MB message, showing that large data

transfers can become a communication bottleneck.

- Therefore, the experiment shows that message size has a significant effect on MPI communication performance, and unnecessary communication should be minimized for better performance.

Answer the following:

1. What is the fundamental difference between OpenMP and MPI?
    - OpenMP is mainly used for shared-memory systems, where multiple threads share the same memory.
    - MPI is used for distributed-memory systems, where each process has its own memory and processes communicate by sending messages.
2. When would you choose MPI instead of OpenMP?
    - MPI is preferred when the application needs to run across multiple computers/nodes or when the system has distributed memory.
    - For example, large scientific simulations can use MPI to divide the work among processes running on different machines.
3. Why is data distribution important in MPI?
    - Each MPI process has its own memory, so data must be divided among processes properly.
    - Good data distribution:
        - Balances the workload.
        - Reduces unnecessary communication.
        - Prevents some processes from remaining idle.
        - Improves overall performance.
4. What is the role of communication in distributed-memory applications?
    - Communication allows MPI processes to **exchange data and coordinate their work**.

      For example:

        - Process 0 → sends data → Process 1
        - Process 1 → sends result → Process 0
    - Without communication, processes cannot exchange information stored in their separate memories.
5. What challenges arise when the number of MPI processes increases?

- Communication allows MPI processes to **exchange data and coordinate their work**.
- For example:
  - Process 0 → sends data → Process 1
  - Process 1 → sends result → Process 0
- Without communication, processes cannot exchange information stored in their separate memories.

6. How could the concepts learned in this assignment be used in the next MPI assignment involving matrix computations?

   - The concepts can be directly applied to **parallel matrix computation**.
   - For example, a large matrix can be divided among MPI processes:
     - Process 0 → rows 0–24
     - Process 1 → rows 25–49
     - Process 2 → rows 50–74
     - Process 3 → rows 75–99
   - Each process performs calculations on its assigned portion. MPI communication can then be used to **distribute matrix data and collect the results**.
   - The concepts of **rank, communicator, data distribution,** MPI_Send()**,** MPI_Recv()**, communication overhead, and deadlock avoidance** will help design the matrix computation efficiently.

Deliverables

Submit:

1. MPI installation and environment details.
2. Source code for all completed tasks.
3. Execution outputs.
4. MPI rank/group/communicator observations.
5. Performance measurements.
6. Communication-time graphs.
7. Answers to discussion questions.
8. AI/HPC case study.
9. Brief conclusion.